



Instituto Tecnológico de Mazatlán

Control de Acceso con RFID por SPI

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Hernández Barrón

Alumnos:

Rosa Guadalupe Romero Navarro

Javier Ramón Quintero Valdez

Gabriel Alejandro Barrón Sarabia

Reporte de práctica: Control de Acceso con RFID por SPI

1. Descripción del proyecto

Esta práctica consiste en comunicar un Arduino UNO R4 WiFi con un lector RFID RC522 mediante el bus SPI, leer el número único (UID) de una tarjeta o llavero, mostrarlo en el Monitor Serie y utilizarlo para simular un control de acceso: si el UID coincide con uno previamente autorizado, se enciende un LED verde durante 2 segundos; si no coincide, se enciende un LED rojo por el mismo tiempo. El sistema debe seguir leyendo tarjetas nuevas en todo momento, incluso mientras un LED permanece encendido.

2. Objetivos

- Comprender el funcionamiento del bus SPI y el rol de cada una de sus líneas (SCK, MOSI, MISO, SS).
- Instalar y utilizar la librería MFRC522 para comunicarse con el lector RC522.
- Leer e identificar el UID de distintas tarjetas y llaveros RFID.
- Implementar un control de acceso simple comparando el UID leído contra uno autorizado.
- Aplicar millis() para controlar el tiempo de encendido de los LEDs sin bloquear la lectura de nuevas tarjetas.
- Verificar experimentalmente el efecto de desconectar una línea del bus SPI (MISO).

3. Herramientas y material utilizado

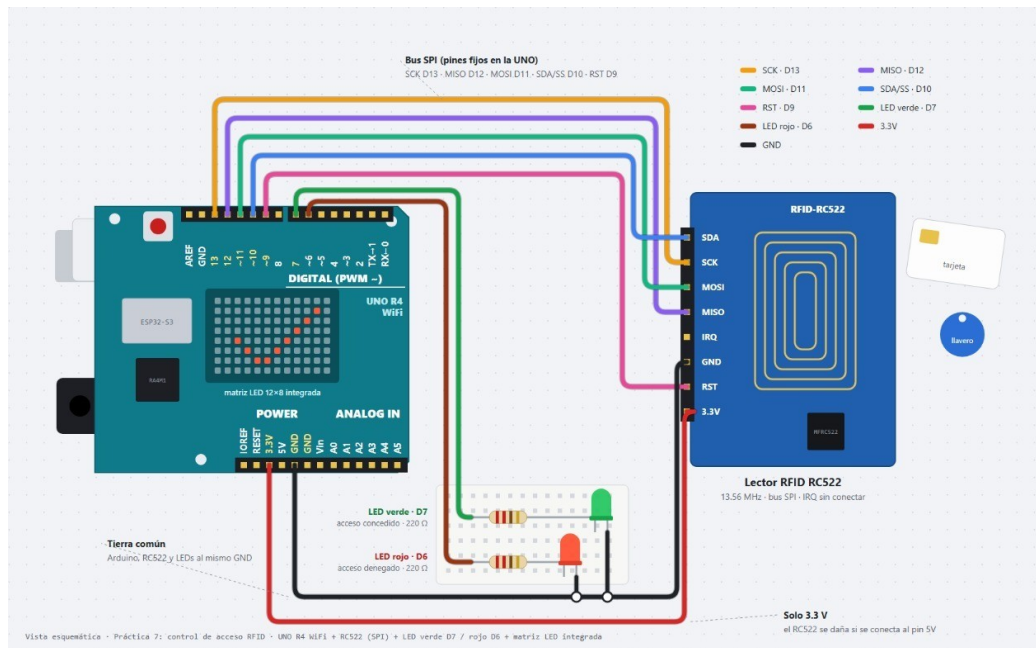
- Arduino UNO R4 WiFi
- Arduino IDE
- Librería MFRC522 (Library Manager)
- Módulo lector RFID RC522
- Tarjeta y llavero RFID
- 2 LEDs (verde y rojo)
- 2 resistencias de 220 Ω
- Protoboard
- Cables de conexión (jumpers)

4. Diagrama

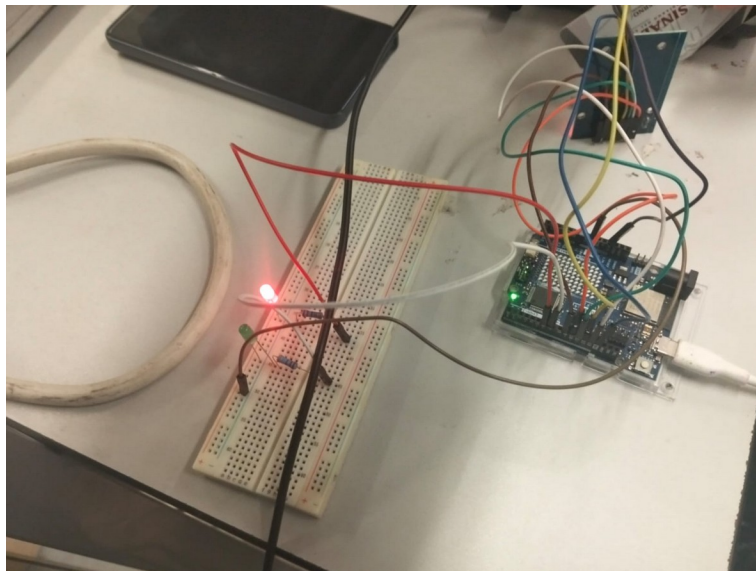
El RC522 se alimenta con 3.3V (no 5V, ya que un voltaje mayor puede dañarlo). Las conexiones utilizadas fueron:

- SDA (SS) → D10
- SCK → D13
- MOSI → D11
- MISO → D12
- RST → D9
- 3.3V → 3.3V
- GND → GND
- LED verde → D7 (con resistencia de 220 Ω a GND)
- LED rojo → D6 (con resistencia de 220 Ω a GND)

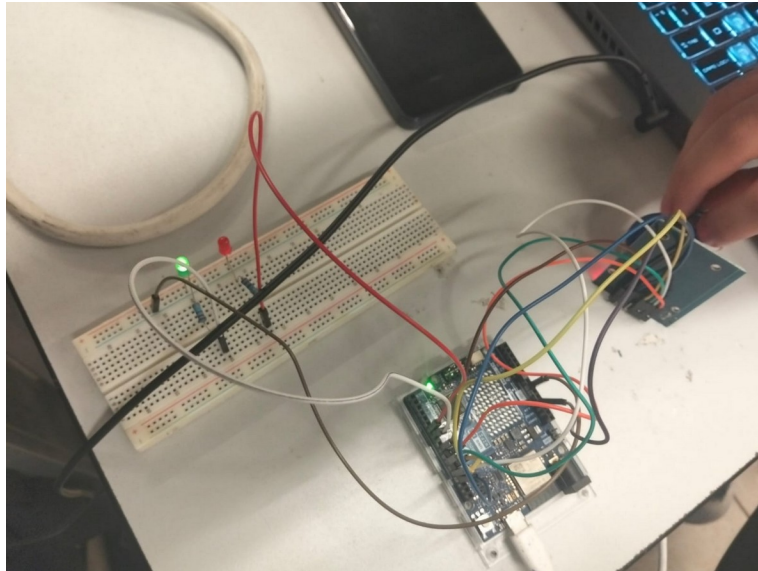
El siguiente esquema muestra la conexión completa del bus SPI entre el Arduino UNO R4 WiFi y el lector RC522, junto con los LEDs de acceso concedido/denegado y la tierra común:



Montaje físico durante la prueba con una tarjeta no autorizada (LED rojo encendido):



Montaje físico durante la prueba con la tarjeta autorizada (LED verde encendido):



5. Metodología

Primero se conectó el módulo RC522 al Arduino UNO R4 WiFi siguiendo el esquema de pines para SPI, cuidando alimentarlo con 3.3V. Se instaló la librería MFRC522 desde el Library Manager del Arduino IDE y se programó una función que verifica la comunicación con el lector leyendo su registro de versión (VersionReg), mostrando un mensaje de confirmación o de error al iniciar.

Posteriormente se programó la lectura del UID de distintas tarjetas, imprimiéndolo en el Monitor Serie en formato hexadecimal separado por espacios. Con el UID de la tarjeta autorizada ya identificado, se guardó dentro del código como un arreglo de bytes, y se implementó la comparación mediante memcmp() para determinar si el acceso debía permitirse o negarse.

Para controlar el tiempo de 2 segundos que cada LED permanece encendido sin bloquear el programa, se utilizó millis() en lugar de delay(), registrando el instante en que se encendió cada LED y verificando en cada vuelta del loop() si ya había transcurrido el tiempo necesario para apagarlo. Finalmente, se realizaron las pruebas indicadas: tarjeta autorizada, tarjeta no autorizada, tarjeta no autorizada con el LED verde aún encendido, y desconexión del cable MISO.

6. Explicación del funcionamiento

6.1 Verificación de comunicación SPI

```
byte version = rfid.PCD_ReadRegister(MFRC522::VersionReg);
```

Al iniciar, el programa lee el registro de versión del chip RC522. Si el valor devuelto es 0x00 o 0xFF, significa que no hay comunicación real por SPI (por ejemplo, si el cableado está mal o una línea está desconectada), y el programa lo reporta como error en lugar de intentar leer tarjetas.

6.2 Lectura y comparación del UID

```
bool uidAutorizado(const MFRC522::Uid &uid) {  
    if (uid.size != TAM_UID_AUTORIZADO) return false;  
    return memcmp(uid.uidByte, UID_AUTORIZADO, TAM_UID_AUTORIZADO) == 0;  
}
```

Cada vez que se detecta una tarjeta nueva, se lee su UID y se compara byte por byte contra el UID autorizado guardado en el código. Si coinciden en tamaño y contenido, el acceso se permite; en caso contrario, se deniega.

6.3 Control de tiempo no bloqueante con millis()

```
if (verdeEncendido && (ahora - inicioVerde >= TIEMPO_LED)) {  
    digitalWrite(LED_VERDE, LOW);  
    verdeEncendido = false;  
}
```

En cada vuelta del loop() se verifica si ya transcurrieron los 2000 ms desde que se encendió cada LED, y de ser así se apaga automáticamente. Como esta verificación no bloquea la ejecución, el programa puede seguir leyendo tarjetas nuevas en todo momento, incluso mientras un LED permanece encendido.

7. Análisis de resultados

Al iniciar el sistema, el lector RC522 fue detectado correctamente mediante la lectura de su registro de versión, confirmando comunicación SPI funcional. Al acercar la tarjeta autorizada, el Monitor Serie mostró "ACCESO PERMITIDO" y el LED verde se encendió, apagándose automáticamente a los 2 segundos. Al acercar una tarjeta no autorizada, se mostró "ACCESO DENEGADO" y se encendió el LED rojo por el mismo tiempo.

Se comprobó también que, con el LED verde encendido tras una lectura válida, el sistema seguía leyendo nuevas tarjetas sin quedarse bloqueado: al acercar una tarjeta no autorizada en ese momento, el Monitor Serie mostró de inmediato "ACCESO DENEGADO", demostrando que millis() permitió que ambas señales (LEDs y lectura de tarjetas) funcionaran de forma independiente.

```
UID: 01 35 D5 1C  
ACCESO DENEGADO  
UID: 01 35 D5 1C  
ACCESO DENEGADO  
UID: 01 35 D5 1C  
ACCESO DENEGADO  
UID: 8B 73 08 89  
ACCESO DENEGADO  
UID: 01 35 D5 1C  
ACCESO PERMITIDO
```

Como se observa en la captura, el sistema detectó repetidamente el UID 01 35 D5 1C como no autorizado en varios intentos consecutivos, hasta que finalmente ese mismo UID fue reconocido como autorizado, mostrando "ACCESO PERMITIDO". También se registró un UID distinto (8B 73 08 89), correspondiente a otra tarjeta, el cual fue correctamente denegado. Al desconectar el cable MISO (D12) y reiniciar el Arduino, el registro de versión leído resultó inválido, y el programa mostró el mensaje de error de comunicación en lugar de intentar leer tarjetas, confirmando que la verificación inicial del bus SPI funcionó como se esperaba.

8. Preguntas

¿Qué significan las siglas SCK, MOSI, MISO y CS?

SCK (Serial Clock) es la señal de reloj que sincroniza cada bit transmitido en el bus SPI. MOSI (Master Out, Slave In) es la línea por la que el maestro envía datos al esclavo. MISO (Master In, Slave Out) es la línea por la que el esclavo envía datos de vuelta al maestro. CS (Chip Select, también llamado SS o Slave Select) es la línea que el maestro usa para indicar cuál esclavo está activo y debe atender la comunicación en ese momento.

¿Por qué el pin del módulo dice "SDA" si no es I2C? ¿Qué función cumple en realidad?

El chip MFRC522 es multiprotocolo: puede operar por I2C, SPI o UART. La etiqueta "SDA" viene heredada del modo I2C, pero en el modo SPI utilizado en esta práctica esa misma pata se reconfigura y funciona como SS/CS (Slave Select), seleccionando al RC522 como el dispositivo con el que el Arduino quiere comunicarse, y no como una línea de datos bidireccional como lo sería un SDA real.

¿Por qué línea viaja el UID del lector al Arduino?

Por MISO, ya que el Arduino es el maestro y el RC522 es el esclavo: toda la información que el lector envía de vuelta, incluyendo el UID leído de la tarjeta, viaja por esa línea.

¿Qué pasó al desconectar MISO y por qué?

El Arduino dejó de poder recibir respuesta del lector. Al leer el registro de versión del chip (VersionReg), se obtuvo un valor inválido (0x00 o 0xFF), que es la señal de que no hay comunicación real por SPI. El programa detectó esta condición y mostró un mensaje de error, en lugar de intentar leer tarjetas sin comunicación real.

¿Por qué no se permitió usar delay() en esta práctica? ¿Qué problema habría causado?

Porque el requisito era que, mientras un LED estuviera encendido (2 segundos), el sistema siguiera leyendo tarjetas nuevas sin bloquearse. Si se usara delay(2000), el Arduino quedaría detenido por completo esos 2 segundos sin poder detectar ninguna tarjeta, fallando justamente la prueba de acercar una tarjeta no autorizada mientras el LED verde permanece encendido.

Si quisieran conectar un segundo lector RC522 al mismo Arduino, ¿qué cables compartirían y cuál tendría que ser diferente?

Compartirían SCK, MOSI, MISO, GND y 3.3V, ya que son el bus común del protocolo SPI. Lo que tendría que ser diferente es el pin SS/CS (Slave Select): cada RC522 necesita su propio pin de selección individual, porque es la única forma en que el Arduino puede indicar con cuál de los dos lectores desea comunicarse en un momento dado.

9. Conclusiones

Gabriel

En esta práctica comprendí cómo funciona el bus SPI y por qué necesita una línea de selección (CS) distinta por cada dispositivo, a diferencia de I2C donde basta con una dirección. Ver que el pin "SDA" del módulo en realidad funcionaba como CS me ayudó a entender que las etiquetas de un componente dependen del modo en que se esté usando, no son fijas.

Javier

Durante esta práctica aprendí la importancia de verificar la comunicación con un periférico antes de intentar usarlo, leyendo el registro de versión del RC522 al inicio. También me quedó claro por qué no se podía usar delay(): con millis() el programa nunca dejó de "escuchar" nuevas tarjetas, incluso mientras procesaba el tiempo de encendido de un LED.

Rosa

Esta práctica me permitió comprender de forma práctica las diferencias entre SPI e I2C, especialmente al ver que un mismo chip puede ofrecer ambos protocolos, pero usando las patas de manera distinta según el modo. Desconectar el cable MISO y ver cómo el programa detectaba el error en vez de fallar en silencio me hizo entender la importancia de validar la comunicación antes de confiar en los datos recibidos.